

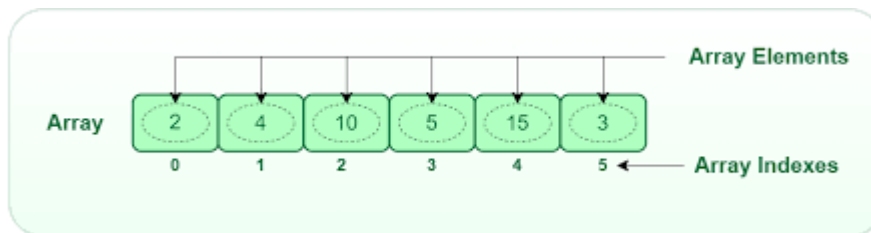
LIST

1. General concept of Lists

Define: A list is a finite collection of elements of the same data type, sorted in a specific order.

Basic mathematical operations:

Initialize the list: Set up an initial empty list and allocate the necessary memory.



With a specific list (array):

- Fixed memory allocation
- Assign the initial number of elements = 0
- Variables can be used to manage the number of existing elements

```
int a[MAX];
```

```
int n = 0;
```

With the following list of links:

- Initialize the head pointer to NULL.
- No nodes have been allocated yet.

```
Node* head = NULL;
```

Meaning:

- This is a mandatory step before any operation.
- Avoid garbage memory access errors (undefined behavior).

Empty check

Array:

```
if (n == 0)
```

Linked List:

```
if (head == NULL)
```

Application:

- Check before deleting, browsing, or retrieving values.

Full check – with arrays only

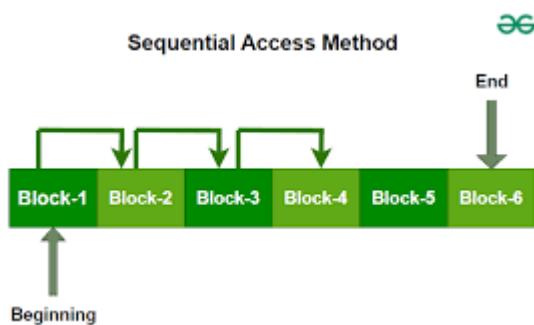
```
if (n == MAX)
```

Complexity: $O(1)$

Note:

- If not checked, memory overflow is likely.
- Linked lists are not needed because of dynamic allocation.

2. Access and search operations



Traverse

Array:

```
for (int i = 0; i < n; i++)
```

Linked List:

```
Node* p = head;
while (p != NULL) {
    p = p->next;
}
```

Complexity: $O(n)$

Application:

- Print list
- Sum, count, and filter data.

Search

Linear Search:

- Applies to both arrays and linked lists.

```
for (int i = 0; i < n; i++)  
    if (a[i] == x)
```

Linked List:

```
while (p != NULL && p->data != x)  
    p = p->next;
```

Complexity:

- Average: $O(n)$

If the array is already sorted:

- Binary Search can be used $\rightarrow O(\log n)$

Get the value

Array:

```
a[i]
```

$\rightarrow O(1)$

Linked List:

- We must traverse to position $i \rightarrow O(n)$

```
for (int k = 0; k < i; k++)  
    p = p->next;
```

Comment:

- This is a major weakness of linked lists.

3. Structure modification operations

Channel (Insert)

With array

Insert at position k:

```
for (int i = n; i > k; i--)  
    a[i] = a[i-1];
```

```
a[k] = x;
```

```
n++;
```

Complexity: $O(n)$

With a Linked List

Insert head:

```
Node* newNode = malloc(...);  
newNode->next = head;  
head = newNode;
```

Insert between:

```
newNode->next = p->next;  
p->next = newNode;
```

Complexity:

- $O(1)$ if there is already a position
- $O(n)$ if the position must be found first.

Delete

With array

```
for (int i = k; i < n-1; i++)
    a[i] = a[i+1];
```

```
n--;
```

Complexity: $O(n)$

With a Linked List

```
p->next = p->next->next;
free(temp);
```

Complexity:

- $O(1)$ if there is already a node
- $O(n)$ if we have to find

Update

Array:

```
a[i] = newValue;
```

Linked List:

```
p->data = newValue;
```

Complexity:

- Array: $O(1)$
- Linked List: $O(n)$ (because nodes need to be found)

General analysis

Mathematical operations	Array	Linked List
Access	$O(1)$	$O(n)$
Point	$O(n)$	$O(1)^*$
Erase	$O(n)$	$O(1)^*$
Search	$O(n)$	$O(n)$